Sprawozdanie - Konrad Żebro, Patryk Persak

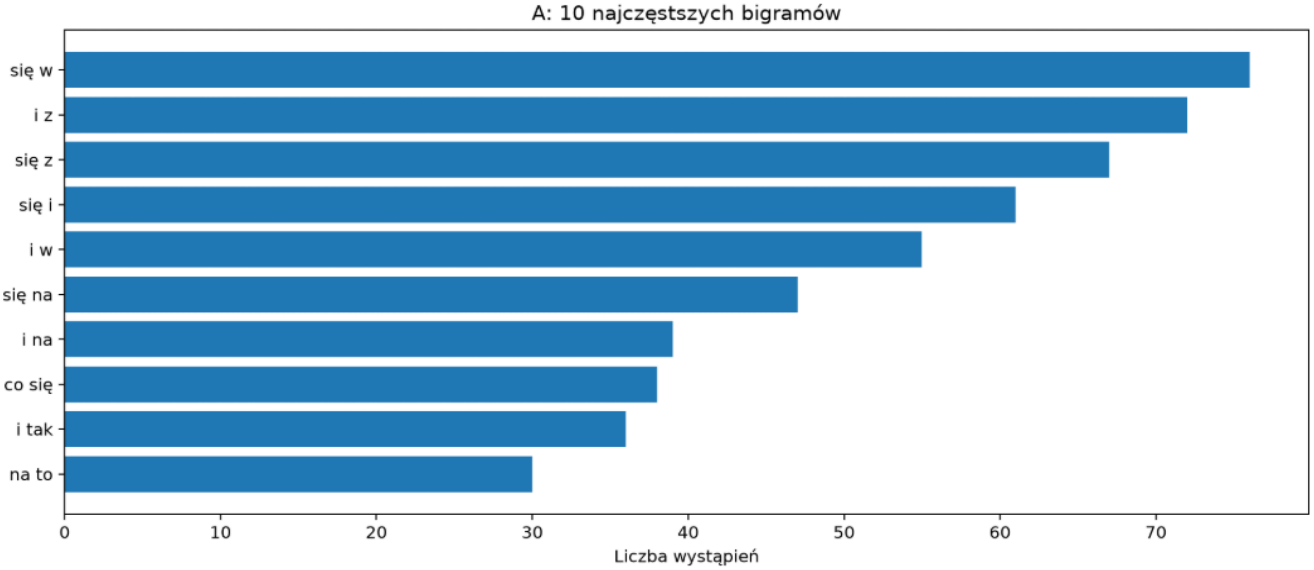
SPRAWOZDANIE

1. Materiał i metody

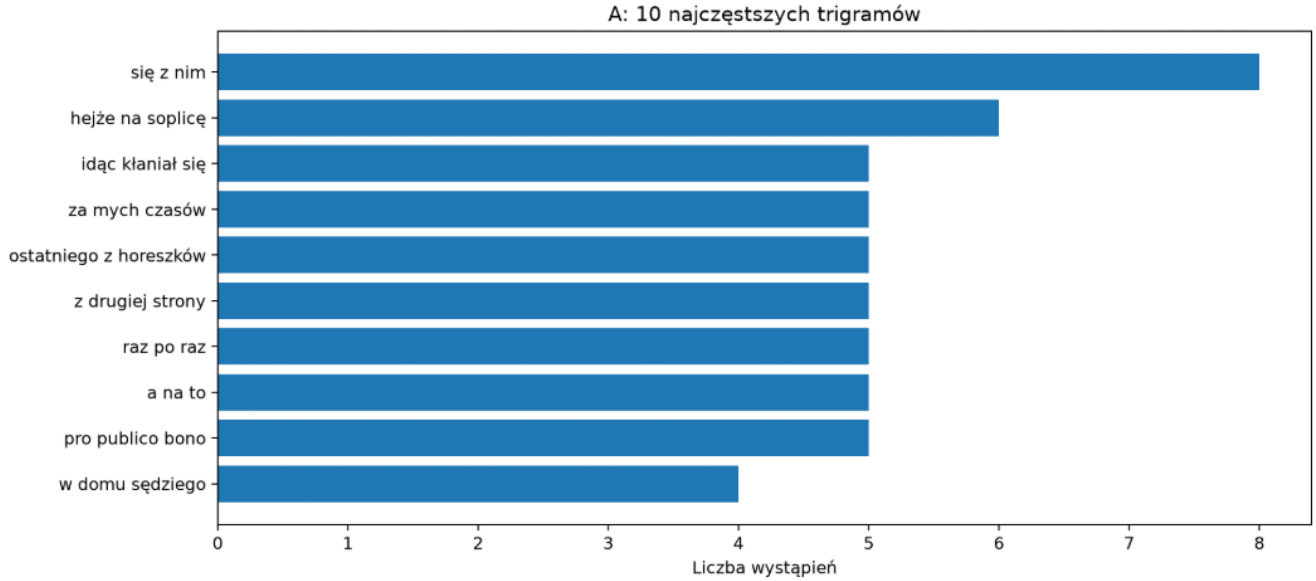
Analizowano „Pana Tadeusza” Adama Mickiewicza z Wolnych Lektur oraz „Hamleta” Williama Szekspira z Project Gutenberg. Z plików tekstowych usunięto nagłówki i stopki serwisów. Tekst polski zamieniono na małe litery, interpunkcję i znaki specjalne zastąpiono spacjami, a wielokrotne białe znaki zwinięto. Polskie znaki diakrytyczne zachowano. Zdania wyodrębniono przed czyszczeniem interpunkcji.

Polski tekst miał 445 325 znaków przed czyszczeniem i 421 597 po czyszczeniu. Uzyskano 3 671 zdań i 68 191 wyrazów. Zliczono 18 760 różnych unigramów, 58 785 bigramów i 67 384 trigramy, korzystając z **collections.Counter**.

2. N-gramy i histogramy



Najczęstsze bigramy to „się w” (76), „i z” (72), „się z” (67), „się i” (61) oraz „i w” (55). Dominują połączenia z zaimkami, spójnikami i przyimkami. Bez usuwania słów funkcyjnych wykres pokazuje więc przede wszystkim częste konstrukcje gramatyczne. Najczęstszy bigram występuje 2,5 raza częściej od dziesiątego (76 wobec 30).



Najczęstszy trigram „się z nim” występuje 8 razy, „hejże na soplicę” 6 razy, a większość pozostałych 4-5 razy. Trigramy pokazują zwroty związane z utworem, np. „za mych czasów” i „w domu sędziego”. Bez porównania z innymi tekstami nie można uznać ich za unikalną cechę stylu autora.

Odpowiedzi na pytania z części A. Dominują bigramy z „się” i słowami funkcyjnymi. Trigramy ujawniają powtarzalne frazy, lecz nie dowodzą wyjątkowości stylu Mickiewicza. Unigramy koncentrują się na najczęstszych wyrazach („i”: 2 126). Bigramy mają więcej typów (58 785 wobec 18 760), niższe częstości i dłuższy ogon rzadkich kombinacji; trigramy są jeszcze bardziej rozproszone.

3. Stemming i lematyzacja angielska

Stemming sprowadza wyraz do rdzenia według reguł obcinania lub zmiany końcówek; rdzeń nie musi być poprawnym słowem, np. **nothing** → **noth**. **Lematyzacja** wybiera podstawową formę słownikową, uwzględniając informacje językowe, często część mowy i kontekst. Pozwala połączyć formy nieregularne, np. **seen** → **see** i **am** → **be**.

Dla pięciu zdań z „Hamleta” użyto tokenizacji NLTK, stemmera Portera oraz WordNetLemmatizer z oznaczeniem części mowy. Osobno porównano stemmery Porter i Lancaster; Lancaster częściej skraca wyrazy agresywnie.

Token	Porter	WordNet	Komentarz
seen	seen	see	Lemat łączy nieregularną formę z czasownikiem.
stood	stood	stand	Lemat rozpoznaje czas przeszły.
am	am	be	Lemat rozpoznaje formę czasownika „be”.
nothing	noth	nothing	Rdzeń Portera nie jest słowem.
graves	grave	graf	Błędne rozstrzygnięcie wieloznaczności.

Wynik **graves** → **graf** pokazuje ograniczenie lematyzatora: wybór formy słownikowej może być błędny w danym kontekście. Pełne wyniki pięciu zdań są w pliku **wyniki/B_token_rdzen_lemat.csv**.

Wyraz	Porter	Lancaster	Ocena
being	be	being	Porter lepiej łączy formę z „be”.
graves	grave	grav	Porter daje czytelniejszy rdzeń.
nature	natur	nat	Lancaster usuwa więcej.
streets	street	streets	Porter usuwa liczbę mnogą.
wonder	wonder	wond	Porter zachowuje całe słowo.

Stemmery dały różne rdzenie dla 1 915 typów wyrazów. **Nadmierne obcinanie:** Lancaster sprowadza „man” i „manner” do „man”, choć mają różne znaczenia. **Niedostateczne sprowadzenie:** Porter pozostawia „see” i „seen” jako różne rdzenie, mimo że to formy tego samego czasownika.

4. Polski stemming i lematyzacja

Dla fragmentu „Pana Tadeusza” o długości 4 500 znaków (682 tokeny) zastosowano polski stemmer Stempel i model spaCy **pl_core_news_sm** do lematyzacji. Następnie te same słowa przetworzono angielskim SnowballStemmer, aby zobaczyć wpływ niedopasowanego języka narzędzia.

Token	Stempel	spaCy	English Snowball
moja	moja	mój	moja
jesteś	y	być	jesteś
dowie	dowie	dowiedzieć	dowi
ozdobie	ozdobie	ozdoba	ozdobi
tobie	ty	ty	tobi
Częstochowy	częstochowy	częstochowy	częstochowi

Stempel potrafi być agresywny (**jesteś** → **y**), a spaCy często daje formę słownikową (**jesteś** → **być**, **moja** → **mój**). Także lematyzacja zależy od modelu i kontekstu. Angielski Snowball zwykle nie zmienia polskich słów albo mechanicznie usuwa końcówki, np. **dowie** → **dowi**, **tobie** → **tobi**, **zdrowie** → **zdrowi** i **ozdobie** → **ozdobi**; nie jest to poprawna analiza polskiej fleksji.

SnowballStemmer w NLTK obsługuje arabski, duński, niderlandzki, angielski, fiński, francuski, niemiecki, węgierski, włoski, norweski, portugalski, rumuński, rosyjski, hiszpański i szwedzki; lista zawiera też wariant „porter”. **Polskiego na niej nie ma.**

5. Odległość edycyjna i inne miary

Levenshtein oblicza minimalną liczbę wstawień, usunięć i zamian pojedynczych znaków. Dla „Ala ma kota” i „Ala posiada kota” odległość wynosi **6**. Jedna optymalna ścieżka zmienia „ma” w „posiada”: pięć wstawień znaków „p”, „o”, „s”, „i”, „a” i jedna zamiana „m” na „d”; pozostałe znaki są dopasowane. Możliwe są inne ścieżki o tej samej długości.

Damerau-Levenshtein dopuszcza dodatkowo zamianę sąsiednich znaków, więc „kot” / „kto” ma odległość 1 zamiast 2. **Jaro-Winkler** uwzględnia dopasowane znaki, transpozycje i wspólny początek. **Jaccard** porównuje zbiory bigramów znakowych jako iloraz liczby elementów wspólnych i liczby elementów w sumie zbiorów. Jaro-Winkler i Jaccard zwracają podobieństwo od 0 do 1. Surowe odległości Levenshteina i Damerau dodatkowo znormalizowano do podobieństwa 0-1.

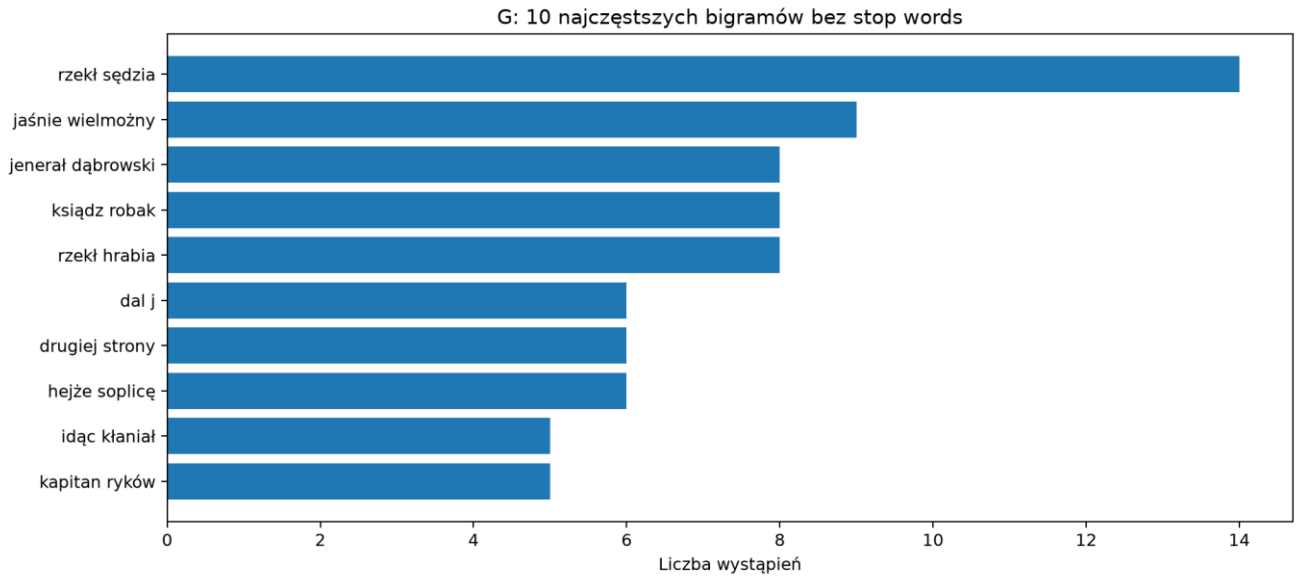
Para	Lev. odl.	Dam. odl.	Lev. pod.	Dam. pod.	Jaro-W.	Jaccard
Ala ma kota / Ala posiada kota	6	6	0,625	0,625	0,847	0,471
kot / kto	2	1	0,333	0,667	0,556	0,000
zolw / żółw	3	3	0,250	0,250	0,500	0,000
Warszawa / Warzawa	1	1	0,875	0,875	0,971	0,625
dom / tom	1	1	0,667	0,667	0,778	0,333

Największa rozpiętość wyników dotyczy **„kot” / „kto”**: od 0 dla Jaccarda do 0,667 dla Damerau. Przestawienie „o” i „t” jest jedną transpozycją, ale oba napisy nie mają wspólnego bigramu znakowego („ko”, „ot” wobec „kt”, „to”). Dla „Warszawa” / „Warzawa” Jaro-Winkler daje wysokie 0,971 dzięki wspólnemu początkowi. Diakrytyki w „zolw” / „żółw” traktowano jako różne znaki. Odległości i podobieństw nie należy porównywać bez normalizacji.

6. Stop words i nazwy własne

Stop words usunięto listą z biblioteki **stop-words** (335 wyrazów; nltk nie ma listy dla polszczyzny). Zniknęło 24 562 z 68 191 wyrazów (36,0%), ale tylko 274 typy, bo stop words to nieliczne, bardzo częste wyrazy. Na pozostałym tekście powtórzono analizę n-gramów z części A.

N-gramy	Wariant	Wystąpienia	Typy	Udział top 10	Trzy najczęstsze
unigramy	ze stop words	68 191	18 760	16,4%	i (2126), w (1693), się (1496)
unigramy	bez stop words	43 629	18 486	2,2%	rzekł (155), hrabia (129), sędzia (127)
bigramy	ze stop words	68 190	58 785	0,8%	się w (76), i z (72), się z (67)
bigramy	bez stop words	43 628	42 713	0,2%	rzekł sędzia (14), jaśnie wielmożny (9), jenerał dąbrowski (8)
trigramy	ze stop words	68 189	67 384	0,1%	się z nim (8), hejże na soplicę (6), idąc kłaniał się (5)
trigramy	bez stop words	43 627	43 480	0,1%	pro publico bono (5), wieku urzędu zaszczyt (4), urzędu zaszczyt należy (4)





Porównanie. Różnice są istotne. Ze stop words top 10 unigramów obejmuje 16,4% tekstu, bez nich 2,2%. W pełnym tekście dominują połączenia funkcyjne („się w", „i z"), które pokazują styl i składnię. Po filtracji wychodzą postacie i sytuacje („rzekł sędzia", „ksiądz robak", „jenerał dąbrowski"), czyli treść. Wariant bez stop words lepiej pokazuje więc treść, a wariant pełny — styl autora.

Filtracja tworzy pary, które w tekście nie sąsiadowały („idąc kłaniał" z „idąc kłaniał się"), a trigramy prawie przestają się powtarzać (43 480 typów na 43 627 wystąpień); najczęstsze pochodzą z jednego, czterokrotnie powtórzonego dwuwiersza. Bigram „dal j" to artefakt czyszczenia z części A: archaiczne „daléj" traci literę é. Dotyczy to kilkudziesięciu wystąpień i nie zmienia wniosków.

Nazwy własne. Wyszukano je w oryginalnym tekście wyrażeniem regularnym:

(?<=[a-ząćęłńóśźż,;])[A-ZĄĆĘŁŃÓŚŹŻ][a-ząćęłńóśźż]+

Część **[A-ZĄĆĘŁŃÓŚŹŻ][a-ząćęłńóśźż]+** dopasowuje wyraz z wielkiej litery. Warunek wsteczny **(?<=[a-ząćęłńóśźż,;])** wymaga przed nim małej litery, przecinka lub średnika i spacji. Dzięki temu pomija początki zdań, wypowiedzi po myślniku i, co w „Panu Tadeuszu" kluczowe, początki wersów, które zawsze zaczynają się wielką literą.

Wyrażenie dało 2 765 trafień (925 form). Część z nich to wyrazy pisane wielką literą z szacunku lub dla podniosłości, dlatego odrzucono formy, które w całym tekście częściej występują małą literą, np. **Pan** (11 razy wielką literą, 128 razy małą), „Waść", „Jego". Zostało 828 form. Najczęstsze:

Lp.	Forma	Wystąpienia
1	Sędzia	89
2	Hrabia	79
3	Wojski	57
4	Tadeusza	50
5	Tadeusz	43
6	Gerwazy	38
7	Sędziego	37
8	Robak	36
9	Litwie	34
10	Ryków	34

Dominują postacie (Sędzia, Hrabia, Wojski, Tadeusz, Gerwazy, Robak, Ryków ex aequo z formą Klucznik) i Litwa. Ograniczenia: odmiana rozbija nazwy na formy (Tadeusz / Tadeusza), tytuły pełnią rolę imion, a nazwy na początku zdania są pomijane. Zliczenie po lematach połączyłoby formy fleksyjne.

7. Własny tokenizator podsłowowy (BPE)

Tokenizator BPE wytrenowano biblioteką **tokenizers** na całym „Panu Tadeuszu" (normalizacja NFC, podział po

spacjach ze znacznikiem _, oddzielona interpunkcja). Słownik startowy obejmuje litery łacińskie i polskie, więc nie ma tokenów nieznanych. Warianty 1 000, 5 000 i 30 000 tokenów osiągnęły zadany rozmiar; wariant 5 000 zapisano jako **tokenizer_pl.json**.

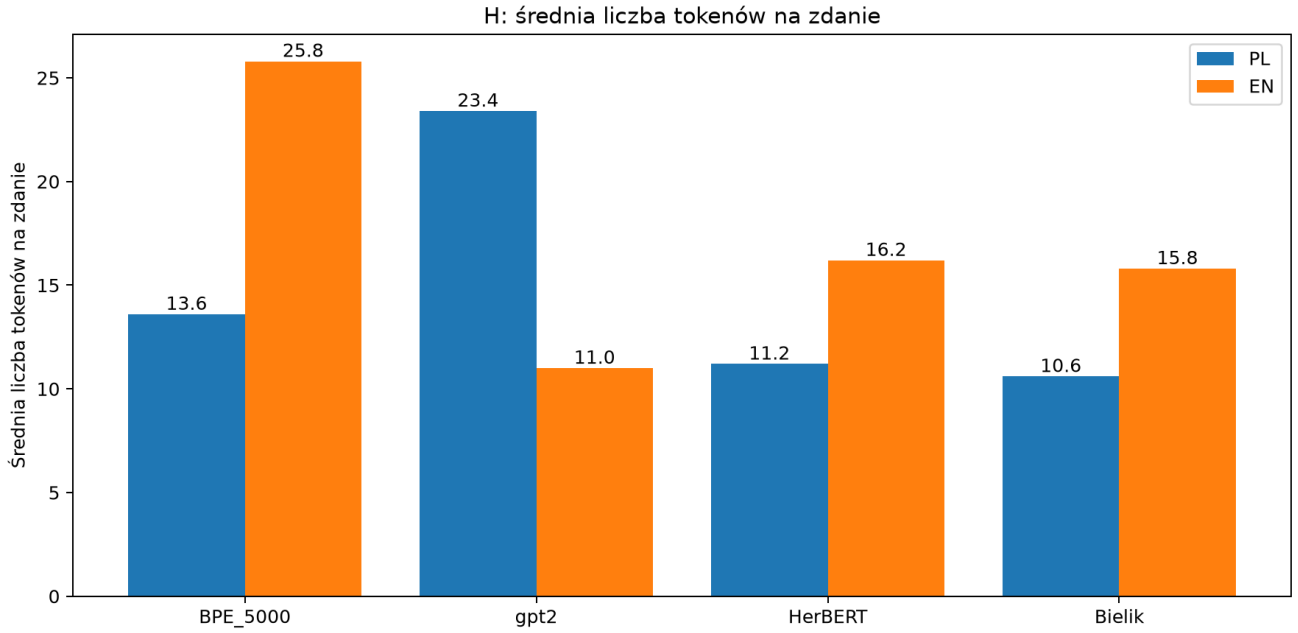
Słowo	BPE 1 000	BPE 5 000	BPE 30 000
nieprzetłumaczalny	_nie | p | rze | t | łu | ma | cza | l | ny (9)	_nie | prze | t | łu | ma | cza | l | ny (8)	_nie | prze | t | łuma | cza | lny (6)
hiperparametryzacja	_h | i | p | er | pa | ra | me | t | ry | za | c | ja (12)	_hi | p | er | pa | ra | me | try | za | c | ja (10)	_hi | per | pa | ra | me | try | za | cja (8)
kotkowi	_ko | t | ko | wi (4)	_kot | kowi (2)	_kot | kowi (2)
Soplicowo	_Soplico | wo (2)	_Soplico | wo (2)	_Soplicowo (1)

Większy słownik daje dłuższe kawałki. Słowa z książki stają się pojedynczymi tokenami (**Soplicowo** przy 30 000), a techniczne rozpadają się na sylaby (**hiperparametryzacja**: 12 → 10 → 8 części). Podział **kotkowi** → **_kot + kowi** przypomina rdzeń i końcówkę.

Do porównania wybrano pięć cytatów z „Hamleta” i ich polskie tłumaczenia (własne), więc obie grupy mają tę samą treść. Tokeny liczono bez tokenów specjalnych. Bielik to tokenizator **speakleash/Bielik-1.5B-v3**.

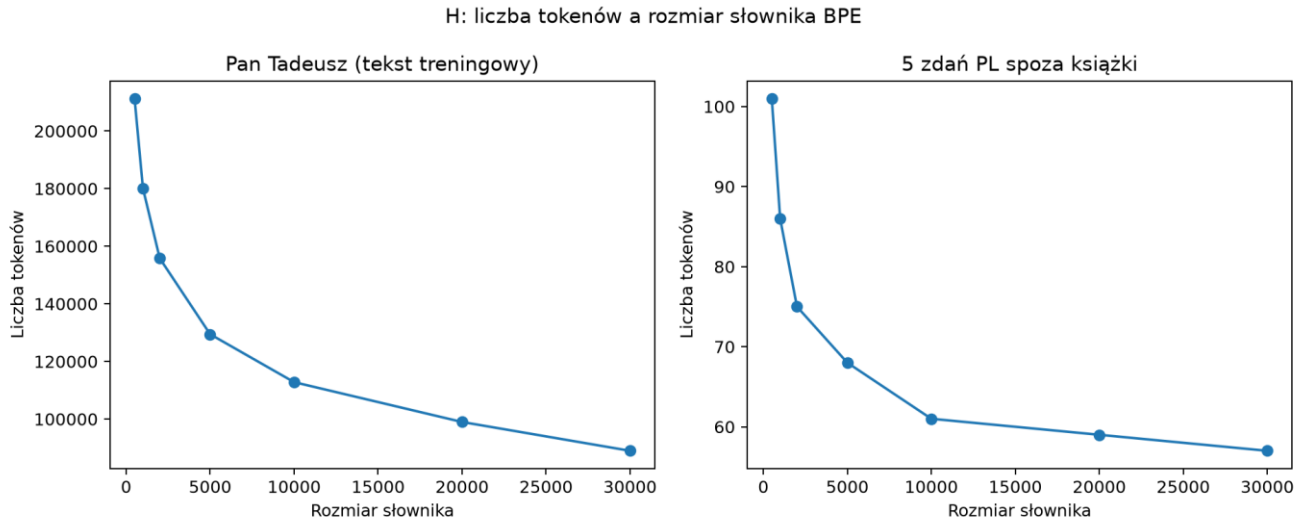
Język	Zdanie	BPE_5000	gpt2	HerBERT	Bielik
PL	Być albo nie być, oto jest pytanie.	11	17	9	9
PL	Słabości, twe imię jest kobieta!	10	19	10	9
PL	Nie bądź ani dłużnikiem, ani wierzycielem.	15	28	10	10
PL	Więcej jest rzeczy na niebie i ziemi, Horacjo, niż się śniło waszym filozofom.	26	43	22	20
PL	Reszta jest milczeniem.	6	10	5	5
EN	To be, or not to be, that is the question.	25	13	15	15
EN	Frailty, thy name is woman!	16	8	11	12
EN	Neither a borrower nor a lender be.	19	8	12	13
EN	There are more things in heaven and earth, Horatio, than are dreamt of in your philosophy.	57	21	35	31
EN	The rest is silence.	12	5	8	8

Tokenizator	Śr. tokenów PL	Śr. tokenów EN	PL / EN	Tokeny na wyraz: Pan Tadeusz	Tokeny na wyraz: tekst współczesny
BPE_5000	13,6	25,8	0,53	1,61	2,48
gpt2	23,4	11,0	2,13	3,30	3,83
HerBERT	11,2	16,2	0,69	1,44	1,11
Bielik	10,6	15,8	0,67	1,49	1,08



Który tokenizator jest najbardziej ekonomiczny? Dla polskiego Bielik (10,6 tokenu na zdanie; HerBERT 11,2), dla angielskiego gpt2 (11,0). gpt2 koduje polskie znaki jako dwa bajty, więc zdanie polskie kosztuje go 2,13 razy tyle tokenów co angielskie. Własny BPE, HerBERT i Bielik faworyzują polszczyznę.

Jak liczba tokenów zależy od rozmiaru słownika? Maleje, ale nieliniowo: każde podwojenie słownika daje 12%–17% mniej tokenów, więc spadek jest w przybliżeniu logarytmiczny. Przejście 1 000 → 5 000 oszczędza 12,7 tokenu na nowy wpis, a 5 000 → 30 000 już tylko 1,6. Na zdaniach spoza książki krzywa wypłaszcza się jeszcze szybciej (68 tokenów przy 5 000, 57 przy 30 000).



Dlaczego tokenizator z jednej książki wypada gorzej na innym tekście? BPE scala pary częste w danych treningowych, więc słownik odzwierciedla jedną książkę: imiona, archaizmy i częste formy wiersza. Wariant 5 000 daje 1,61 tokenu na wyraz na „Panu Tadeuszu" i 2,48 na współczesnym akapicie (**smartfonie** → **_s|m|art|f|o|nie**), a wariant 30 000 prawie zapamiętuje książkę (1,03 wobec 1,86). HerBERT i Bielik, uczone na dużych korpusach, mają odwrotnie (1,44 / 1,11 i 1,49 / 1,08).

Związek z podatkiem językowym. Podatek językowy to dodatkowe tokeny za tę samą treść w języku słabiej reprezentowanym w danych tokenizatora: wyższy koszt, wolniejsze działanie, mniej treści w oknie kontekstu. gpt2 potrzebuje dla polskiego 2,13 razy tyle tokenów co dla angielskiego; tokenizatory uczone na polszczyźnie odwracają ten stosunek (BPE_5000 0,53, HerBERT 0,69, Bielik 0,67). Koszt tokenizacji zależy więc od danych treningowych tokenizatora, a nie od samego języka.

8. Porównanie metod przetwarzania

Fragment z części E (4 500 znaków) przetworzono czterema sposobami: same wyrazy, lematy spaCy, wyrazy bez stop words i tokeny BPE 5 000. Słownik to liczba unikalnych jednostek; dla skali pomiar powtórzono na całej książce.

Wariant	Jednostki (fragment)	Słownik (fragment)	Zmiana słownika	Jednostki (książka)	Słownik (książka)	Zmiana słownika
same wyrazy	682	492	+0,0%	68 144	18 761	+0,0%
wyrazy po lematyzacji (spaCy)	682	435	−11,6%	68 144	13 461	−28,3%
wyrazy bez stop words	451	418	−15,0%	43 585	18 487	−1,5%
tokeny BPE 5000	1 180	726	+47,6%	109 897	4 837	−74,2%

Wariant	Początek fragmentu
same wyrazy	litwo ojczyzno moja ty jesteś jak zdrowie ile cię trzeba cenić ten
wyrazy po lematyzacji (spaCy)	litwo ojczyzno mój ty być jak zdrowie ile ty trzeba cenić ten
wyrazy bez stop words	litwo ojczyzno jesteś zdrowie cenić dowie stracił piękność twą całej ozdobie widzę
tokeny BPE 5000	_Lit wo _Oj czyz no _moja _ty _jesteś _jak _zdrowie _Ile _cię

Na fragmencie słownik najbardziej zmniejsza usunięcie stop words (−15,0%), ale w książce tylko o 1,5%, bo usuwa 274 typy; skraca tekst, a traci składnię i negację. Lematyzacja łączy formy fleksyjne (−11,6% i −28,3%)

kosztem informacji gramatycznej i błędów (**litwo**, **ojczyzno** bez zmian). BPE na fragmencie powiększa słownik (+47,6%), ale w książce ma najmniejszy: 4 837 jednostek (−74,2%), z góry ograniczony do 5 000 i bez słów spoza słownika.

Wniosek: słownik najmocniej zmniejsza tokenizacja podsłowowa, kosztem o 61% dłuższych sekwencji i jednostek bez znaczenia (▁Lit wo, ▁Oj czyz no). Wśród metod działających na wyrazach najwięcej daje lematyzacja; usuwanie stop words skraca tekst, ale słownika prawie nie zmienia.

9. Wnioski

Dłuższe n-gramy występują rzadziej i tworzą większy zbiór unikalnych kombinacji. Stemming jest prosty, ale może obcinać za mocno lub za słabo; lematyzacja daje formy słownikowe, lecz może błędnie rozstrzygać znaczenie. Do polskiego potrzebne są narzędzia uwzględniające jego fleksję. Miary podobieństwa reagują inaczej na liczbę edycji, przestawienie znaków, wspólny początek i wspólne pary znaków.

Usunięcie stop words przesuwa n-gramy ze stylu na treść, ale słownika prawie nie zmniejsza. Wyrażenie regularne z warunkiem na poprzedzający znak dobrze znajduje nazwy własne. Tokenizacja podsłowowa daje mały, zamknięty słownik kosztem dłuższych sekwencji, a jej koszt zależy od danych treningowych tokenizatora; stąd podatek językowy polszczyzny w gpt2.

Materiały

Teksty: **data/pan-tadeusz.txt**, **data/hamlet.txt**. Kod: **lab2.py**, **lab2_impl.py**. Pełne wyniki: pliki CSV i PNG w katalogu **wyniki**. Źródła tekstów: Wolne Lektury, „Pan Tadeusz”; Project Gutenberg, „Hamlet”.

Część na 5.0: **stop-words**; tokenizatory Hugging Face: **gpt2**, **allegro/herbert-base-cased**, **speakleash/Bielik-1.5B-v3**; własny tokenizator **tokenizer_pl.json** (5 000 tokenów). Wyniki G–I: pliki **G_**, **H_**, **I_** w katalogu **wyniki** oraz **wyniki/wyniki_lab2.txt**.